Assignment 3 Charis Aletraris

Exercise 1: Recurrent Neural Networks

Question 1: Feedforward networks and RNNs

What is a feedforward neural network

A feedforward neural network learns a mapping from an input to an output. Information moves in one direction: from the input, through hidden layers, to the prediction. There is no loop carrying information from a previous input into the next one. The previous inputs are used just to train the parameters of the hidden layer (if the model is still learning).

Feedforward networks are used for tasks such as predicting prices from numerical features, classifying customer records and recognizing handwritten digits from pixel values. They are useful when the information needed for a prediction is supplied together as a fixed-size input.

What is a recurrent neural network

A recurrent neural network (RNN) processes a sequence one step at a time. At each step, it uses the current input and a hidden state from the previous step. The hidden state acts as a short summary of what the network has read so far. The same weights are reused at every step.

For example, an RNN reads a movie review word by word. In "The movie was not good", it can learn to carry information about "not" forward when it reaches "good". This helps it predict the meaning of the whole review instead of relying on one word alone.

RNNs are used for sequential data, including text, speech and measurements collected over time.

Main Difference	
Feedforward network	**RNN**
Processes the supplied input through its layers	Processes a sequence step by step
Does not carry a hidden state between inputs	Carries a hidden state between sequence steps
Usually expects a fixed-size input	Can process sequences of different lengths
Has no built-in mechanism for remembering earlier sequence steps	Is designed to use information from earlier steps

Question 2: The vanishing gradient problem

What it means

During training, a network measures its prediction error and works backward to calculate how its weights should change. A gradient tells it how changing a weight would affect the error. If this gradient becomes extremely small, the learning signal is weak and the weight may change very little. This is the vanishing gradient problem.

Why it happens in RNNs

In an RNN, the learning signal travels backward through the sequence steps. This is called backpropagation through time. The calculation repeatedly multiplies values. If these multiplications keep shrinking the signal, it can become almost zero before reaching the early steps.

How it affects training

The RNN gives more weight to the relationships between nearby information. That has resulted in struggles to learn how information near the beginning of a sequence affects a prediction at the end. For example, a long review may begin with "I did not enjoy this film" and later describe many details. The model may struggle to learn to preserve that early opinion for its final sentiment prediction.

This problem comes to reduce the LSTMs and GRUs by using gates that control how information is kept and updated.

Question 3: Where RNNs are used

Sensor signals

RNNs are useful for analysing sensor signals because these measurements form sequences over time. A single reading may provide limited information, while a sequence can reveal a movement, an activity or a developing fault.

For example Zhu, Chen and Brown (2018) developed a sequence-to-sequence approach for recognising daily activities from motion and environmental sensor data. This demonstrates the usefulness of analysing sensor sequences for monitoring daily activities and supporting older adults living independently.

Speech recognition

Speech recognition converts spoken audio into written words. Audio is processed as a sequence of short segments. An RNN can use information from earlier segments to help interpret the current sound, rather than treating each segment independently.

This is useful because a sound that is unclear on its own may be easier to identify from nearby sounds. RNN-based models, especially LSTM variants, have been effective in speech recognition because they can learn context across the audio sequence.

Question 4

Instead of running only the RNN model I desire to run the 3 different types of model in order to be able to compare the result of those models.I compared SimpleRNN, GRU and LSTM networks with 1, 2 and 3 recurrent layers. Each of the nine experiments ran for 15 epochs on the same data split. I stop running the code at the 15 epochs because I see that the best results in all cases are achieved at the early epochs and in all cases after the epochs 5 I start having an over fidding. The best results have been achieved with the 3-layer GRU: at epoch 3 it achieved both the highest validation accuracy, 89.38%, and the lowest validation binary cross-entropy, 0.2724, across the nine experiments. Test results were not used for this choice.

Best accuracy reached by each model family

Model	Layers	Epoch	Validation accuracy	Loss at that epoch
SimpleRNN	1	4	87.72%	0.3522

Model	Layers	Epoch	Validation accuracy	Loss at that epoch
GRU	3	3	89.38%	0.2724
LSTM	2	2	88.90%	0.2727

Each row shows the highest validation accuracy in that family and the loss measured at the same epoch. It does not combine accuracy from one epoch with loss from another. For SimpleRNN, its lowest loss was instead 0.3248 at epoch 3, with 86.42% accuracy. The two class labels remain negative and positive; layers describe the depth of the network.

Main values shared by all experiments

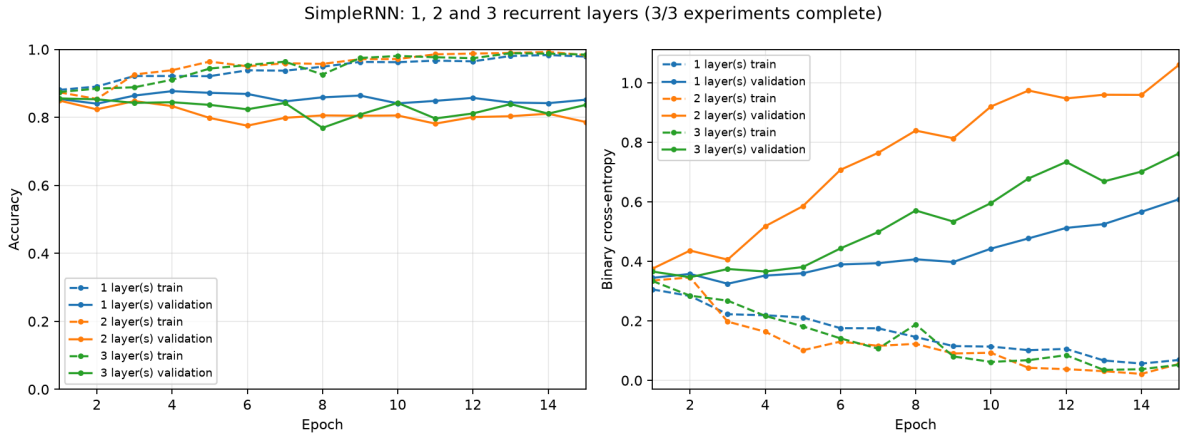
Setting	Value	Purpose
Data split	70% / 10% / 20%	35,000 training, 5,000 validation and 10,000 test reviews; balanced classes and seed 42.
Input	10,000 IDs; 500 tokens	Uses the assignment vocabulary limit and review length; retains the last 500 tokens.
Embedding	32 dimensions	Learns a compact vector for each word ID; zero padding is masked.
Recurrent layers	1, 2 or 3; 32 units each	Changes model type and depth while keeping the width fixed.
Training	15 epochs; batch size 128	Gives every candidate the same epoch budget and batch size.
Optimizer	Adam; learning rate 0.001	Uses adaptive weight updates with a conventional starting learning rate.
Gradient clipping	Norm limit 1.0	Limits excessively large updates; it does not solve vanishing gradients.
Output and loss	1 sigmoid; binary cross-entropy	Predicts positive sentiment probability; threshold 0.5 gives the class.

These are practical starting values, not proven optimal settings. No dropout or extra regularization was added. Parameter counts differ between models, so this compares architectures at fixed width and training settings, not at equal capacity or equal computation.

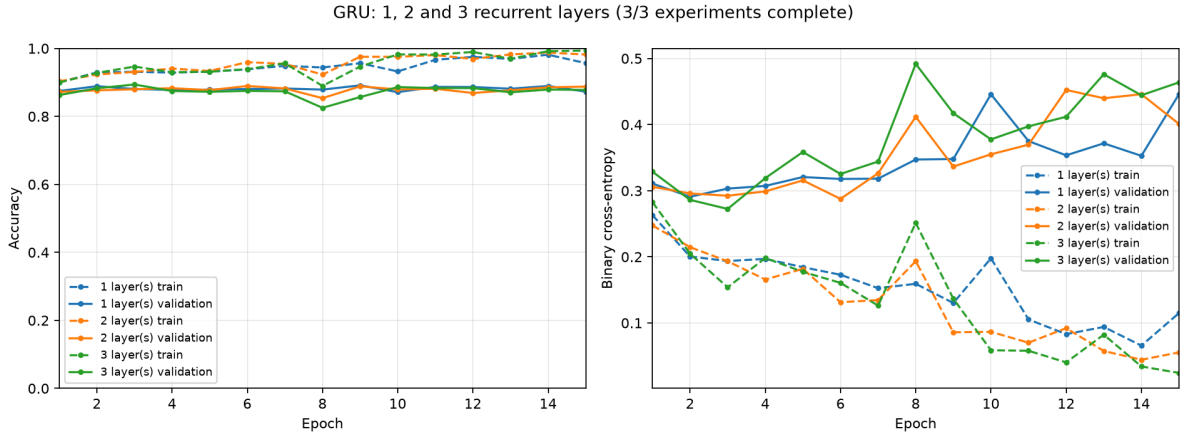
Results for every model and depth

Model	Layers	Parameters	Best accuracy	Epoch	Related loss
SimpleRNN	1	322,113	87.72%	4	0.3522
SimpleRNN	2	324,193	84.96%	1	0.3752
SimpleRNN	3	326,273	85.58%	1	0.3656
GRU	1	326,369	89.12%	9	0.3479
GRU	2	332,705	88.94%	6	0.2877
GRU	3	339,041	89.38%	3	0.2724
LSTM	1	328,353	88.54%	3	0.2939
LSTM	2	336,673	88.90%	2	0.2727
LSTM	3	344,993	88.66%	3	0.2898

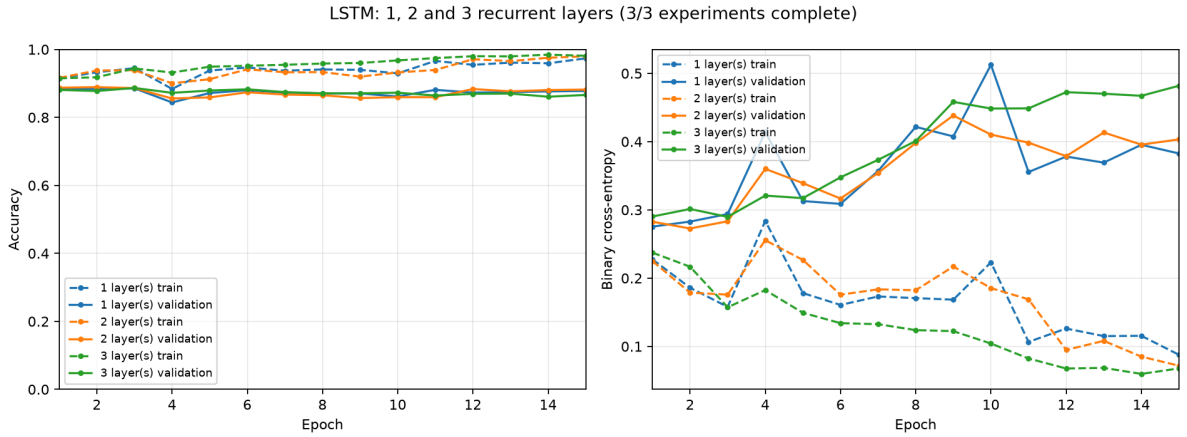
Comparison learning curves



SimpleRNN: training and validation accuracy and loss for 1, 2 and 3 recurrent layers. Dashed lines show training; solid lines show validation.



GRU: training and validation accuracy and loss for 1, 2 and 3 recurrent layers. Dashed lines show training; solid lines show validation.



LSTM: training and validation accuracy and loss for 1, 2 and 3 recurrent layers. Dashed lines show training; solid lines show validation.

Source: results_recurrent_comparison/20261015_150230_305523, histories.json, config.json and the three comparison figures. Each curve uses end-of-epoch evaluation with the same current weights for training and validation.

Overall conclusion

We can easily identify that for all models we have the best accuracy in the early layer. Also, as we expected the simple RNN has the lowest accuracy. We accept those results because the simple RNN is struggling to retain information across long reviews because the learning signal can become very small. GRU and LSTM networks use gates that help preserve useful context. It is therefore reasonable to expect gated networks to help with this task.

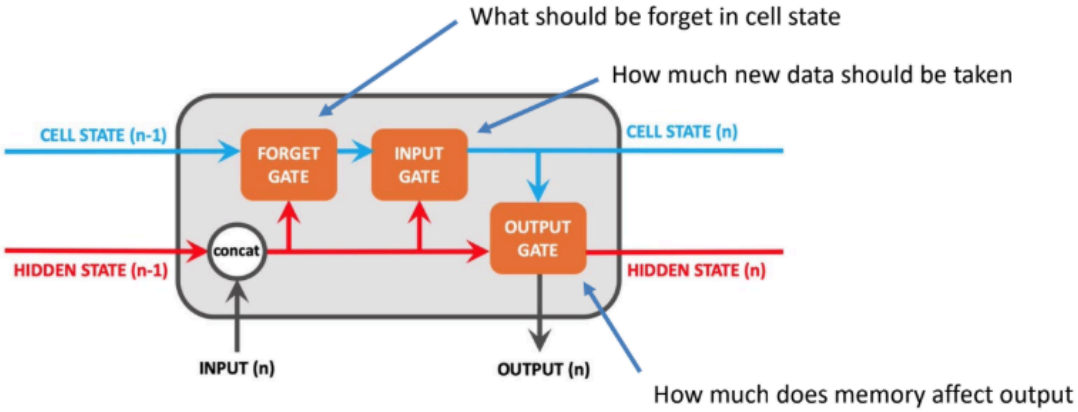
Regarding the time we can see a noticeable difference between the simple RNN with the LSTM and GRU, which needs almost half the time per epoch. The time for GRU and LSTM is very similar.The 3-layer GRU can process the word sequence through several stages, allowing later layers to combine patterns learned by earlier layers. The comparison supports this depth for the GRU on this split. More layers are not always better: the 1-layer SimpleRNN and 2-layer LSTM gave the best accuracy within their families. Extra layers can also make optimization harder or increase overfitting.

Exercise 2: Long Short-Term Memory Networks

Question 1: How LSTMs reduce vanishing gradients

A separate memory path

An LSTM is a type of RNN designed to learn relationships across longer sequences. In a standard RNN, the learning signal can become very small as it travels backward through many steps. An LSTM adds a cell state, which acts as a memory carried from one step to the next. This gives information and the learning signal a more direct path through the sequence.



The three gates

The LSTM cell controls information using three gates. Each gate learns values between 0 and 1 from the current input and the previous hidden state. A value close to 0 blocks information, while a value close to 1 allows it through.

Input gate: controls the intake of new information. It decides how much of the new candidate information is added to the cell state.

Forget gate: determines which parts of the previous cell state are kept or removed. In this way, it helps control which old information is updated; the input gate controls what new information is added.

Output gate: determines which information from the updated cell state is exposed as the hidden state. This hidden state is passed to the next step and can be used to make a prediction.

Why this helps training

The memory update can be described simply as: new memory = retained old memory + selected new information. The addition allows the network to carry useful memory forward without repeatedly replacing it with a new transformed hidden state. When the forget gate stays close to 1, the gradient along this memory path can remain strong across many steps. This helps the network learn how an early input affects a later prediction.

Question 2: Applications where LSTMs outperform standard RNNs

Speech recognition

Speech is a sequence of sounds, and an unclear sound may be easier to identify using the sounds around it. LSTMs can retain useful context across the sequence. Graves and Schmidhuber (2005) compared LSTMs and standard RNNs on speech sound classification using the TIMIT dataset. Their bidirectional LSTM achieved 69.8% test accuracy, compared with 69.0% for a bidirectional standard RNN. Bidirectional means that both models read the sequence in both directions. This is a measured improvement for that experiment, rather than a guarantee for every speech task. [1]

Language modelling and next-word prediction

A language model predicts the next word from earlier words. Information near the beginning of a sentence can help determine which word should come later. LSTMs can learn to keep that information for longer than a standard RNN. Sundermeyer, Schluter and Ney (2012) reported about an 8% relative reduction in perplexity compared with standard RNN language models on English and French tasks. Perplexity measures how well the model predicts a word sequence; a lower value means better predictions. This is not an 8 percentage point increase in accuracy. [2]

Connection to our IMDB experiment

Our comparison also showed an advantage for LSTM over SimpleRNN: the best LSTM reached 88.90% validation accuracy, compared with 87.72% for SimpleRNN. However, the three-layer GRU reached 89.38%, so LSTM was not the best model in our experiment. The best choice depends on the task, data and settings.

References

[1] Graves, A. and Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM and other neural network architectures. Neural Networks, 18(5-6), 602-610. https://doi.org/10.1016/j.neunet.2005.06.042

[2] Sundermeyer, M., Schluter, R. and Ney, H. (2012). LSTM neural networks for language modeling. Interspeech 2012, 194-197. https://doi.org/10.21437/INTERSPEECH.2012-65

Question 3: Reuters classification with LSTM networks

Task and model

I trained three separate LSTM models with 1, 2 and 3 recurrent layers for 15 epochs each. Each model reads a Reuters news article and predicts one of 46 topics. This is text classification: the sequence is the words in an article, rather than measurements used to predict a future time value. The one-layer LSTM gave the lowest validation loss and was selected for the final evaluation.

Data split

The 11,228 articles were divided into 7,859 training, 1,123 validation and 2,246 test articles, approximately 70%, 10% and 20%. I kept the original Keras test set and reserved 12.5% of the remaining articles for validation. The training/validation split was stratified to approximately preserve the proportion of each topic. Training updates the weights, validation selects the model, and test data measures its performance. All three models used the same split.

Why these parameter values were chosen

Vocabulary and input length: I kept the 10,000 most frequent word IDs, as in the assignment, to limit the vocabulary size. As we know we have to keep the sequence in the same size of tokens for the LSTM. The exercise is mentioned to use only 20 tokens, but in my opinion it shows a small size for the token and is possible to lose useful information. For this reason I have made the experiment with 2 options of tokens 20 and 200 in order to be able to compare the results. I also use padding="post" in order to include zeros at the end if the total sequence is less than the selected size. Keeping the beginning is a practical choice because news articles often introduce their topic early; it may lose useful information later in a long article. Short articles are padded on the right, and masking makes the LSTM ignore padding.

Embedding and LSTM size: The embedding represents each word with 64 learned values, and each recurrent layer has 64 units. These are moderate starting sizes that provide capacity to learn word and sequence patterns. Comparing 1, 2 and 3 layers tests whether additional processing improves the result. The intermediate layers return the full sequence so the next LSTM can read it; the final LSTM returns one vector for classification.

Dropout and output: Dropout of 0.2 after the last LSTM randomly removes 20% of its output features during training to help reduce overfitting. Dropout is disabled during evaluation. The final layer has 46 outputs with softmax, giving one probability per topic. Sparse categorical cross-entropy is used because each article has one integer topic label.

Results and choice of model

The table compares the checkpoint with the lowest validation loss for each depth. Accuracy values are taken from that same epoch. Test accuracy is included for reporting only; it did not control the choice of depth or checkpoint.

LSTM layers	Selected epoch	Validation loss	Validation accuracy	Test accuracy
1-200tokens	10	1.3587	68.39%	65.58%
2-200tokens	14	1.4902	63.13%	61.49%
3-200tokens	14	1.4545	64.74%	63.31%
1-20tokens	9	1.3309	70.97%	69.68%
2-20tokens	6	1.3643	68.03%	66.52%
3-20tokens	11	1.4395	67.76%	67.76%

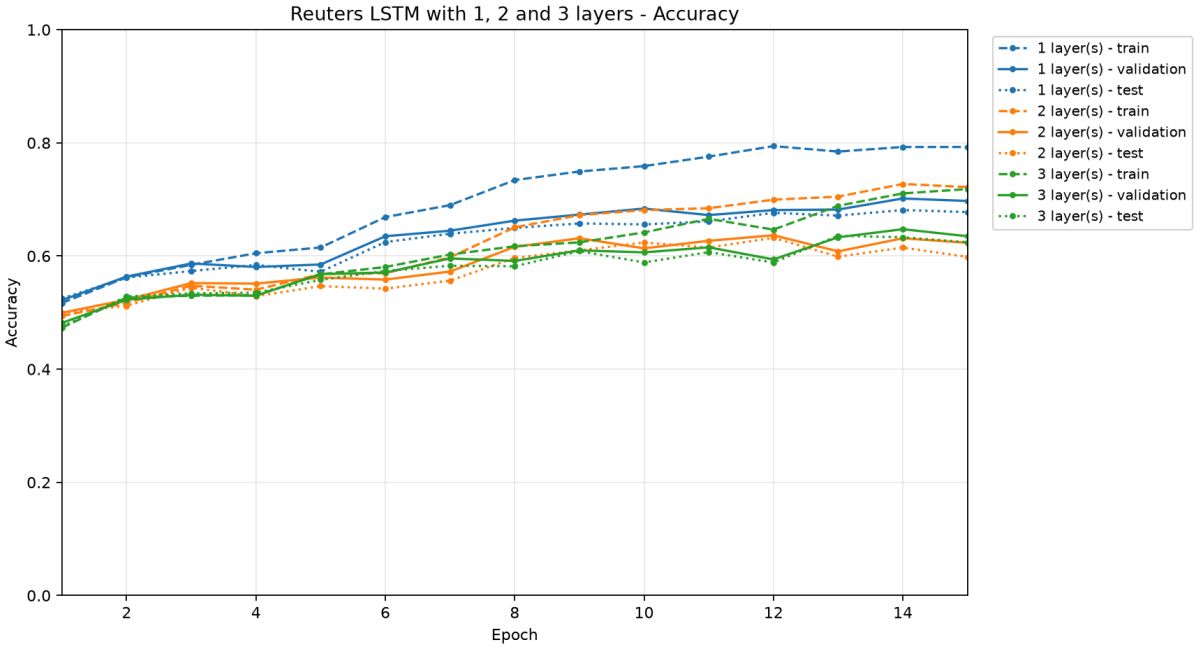
Selected model

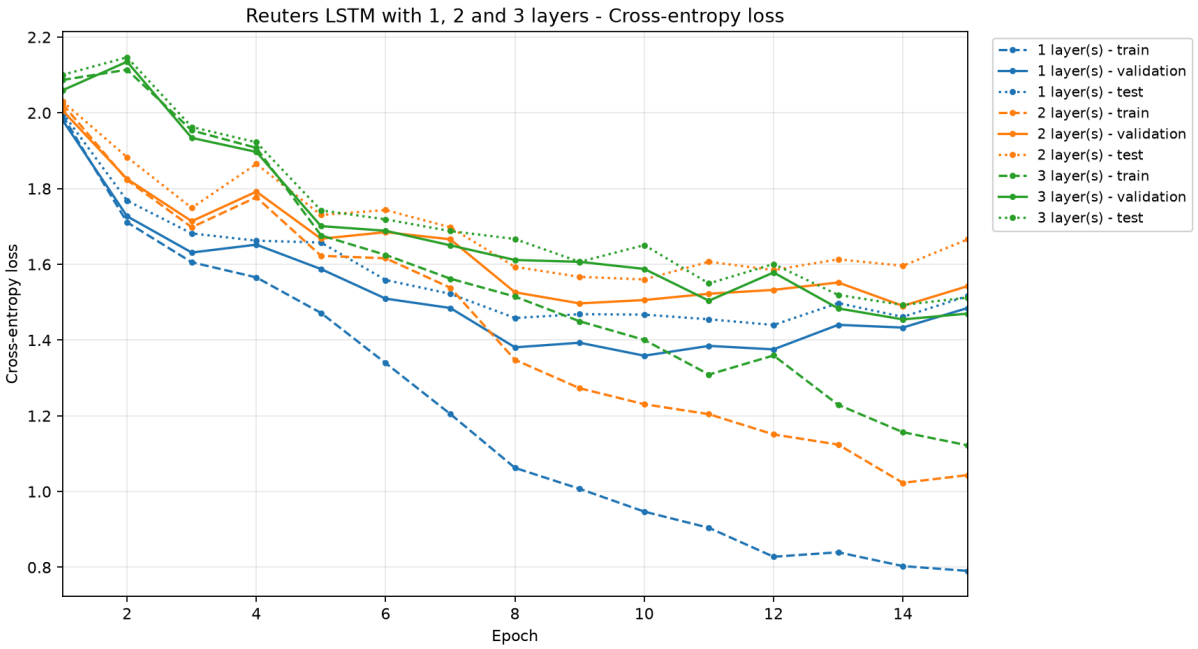
Un expectantly we can easily identify that the results with 20 tokens are much better than the results with 200tokens. This is because the additional information that included in the 200 tokens as most probably is not any useful information that helps my model to find the correct results. That proves that not always the more token means better results. Also we can identify a significant difference in the time that is needed for 20 tokens comparing the time of 20 tokens.

Regarding the layers we can identify that in both cases tThe one-layer LSTM achieved better lower validation lost and highest accuracy in both number of tokens. That also proves that the inclusion of more layers can increase the over feeding in some cases instead of helping to achieve better results. As well increase with the complexity and the time that is needed.

From the one-layer of 200 tokens model we can once identify that the accuracy and loss do not always choose the same epoch.The highest validation accuracy for the one-layer of 200 tokens model was 70.17% at epoch 14, whereas the lowest validation loss occurred at epoch 10. Accuracy only checks whether the predicted topic is correct. Cross-entropy also considers the predicted probabilities and penalizes confident mistakes. Therefore, accuracy can improve slightly while loss becomes worse. I kept the original selection rule of lowest validation loss.

Reuters learning curves of 200 tokens

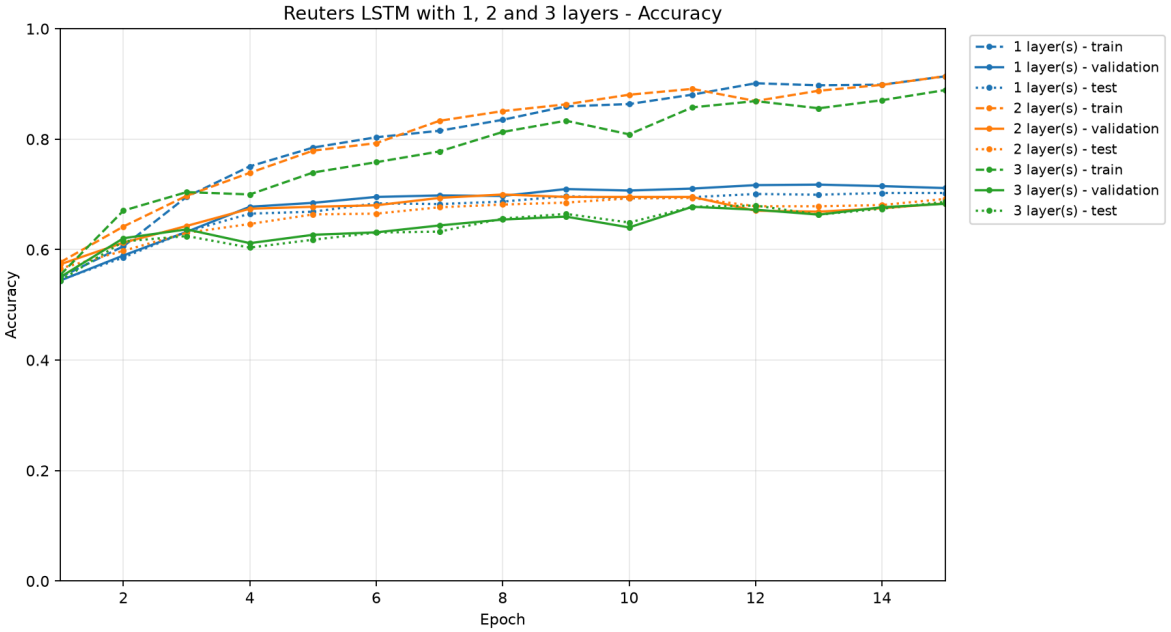


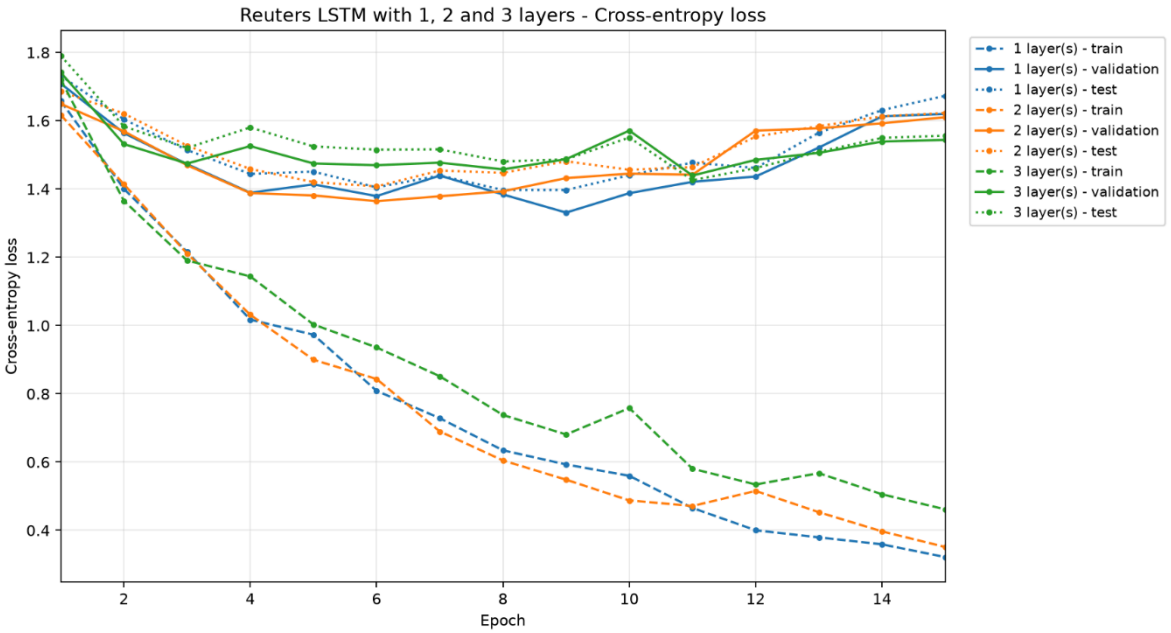


Blue: 1 layer; orange: 2 layers; green: 3 layers. Dashed lines show training, solid lines validation and dotted lines test. All curves use end-of-epoch evaluation, with dropout disabled, so the measurements are comparable.

Source: results_reuters_lstm/20261025_210925_742240. Settings and results are taken from config.json, history.json, summary.json and selected_model.json. All three models completed 15 epochs.

Reuters results with 20 tokens





Blue: 1 layer; orange: 2 layers; green: 3 layers. Dashed lines show training, solid lines validation and dotted lines test. All curves use end-of-epoch evaluation, with dropout disabled. Source: results_reuters_lstm/20tokens; metrics from config.json, history.json, summary.json and selected_model.json.